During this assignment, AI demonstrated significant biases and limitations when analyzing the ESHOP system. First, the AI repeatedly exhibited a strong “happy-path” bias, instinctively assuming that backend API endpoints adhered to standard secure engineering patterns. Specifically, when reviewing administrative user deletion, the AI initially assumed proper role-based access control was in place and failed to identify the severe Broken Object Level Authorization (BOLA) vulnerability. Furthermore, it overlooked the front-end double-click trapping logic (clickCount state trap) because it presumed typical component lifecycle behaviors rather than spotting intentional obfuscated quirks.

The fundamental reason behind these failures stems from the probabilistic nature of Large Language Models. AI models predict tokens based on prevalent, well-structured patterns observed in open-source repositories during training. Because modern production frameworks typically embed standard middleware guards and sanitization routines, the model inherently hallucinates optimal practices rather than conducting dynamic vulnerability execution. Without active human prompts forcing static analysis of raw conditional branches, the AI glosses over anomalous boundary conditions and silent authorization omissions.

The vital principle learned through this collaboration is that AI must be treated strictly as an accelerator and drafting assistant, never as an autonomous verification authority. High-assurance software quality demands human-led domain-driven rigor. Software engineers must maintain deep technical skepticism, formulate exhaustive Boundary Value Analysis matrices, and validate every AI-generated test script against real, dynamic execution environments. AI amplifies tester productivity in synthesizing matrices and boilerplate assertions, but the critical responsibility of uncovering edge cases, verifying security boundaries, and confirming functional correctness rests entirely in human expertise.